

04 — Train Model 2: Hybrid (RoBERTa EN + PhoBERT VI) + R-Drop

Chạy thử 4. Input: faidset-processed + augmented-data. Output: hybrid-rdrop dataset.

Cải tiến so với baseline:

- Dùng train_aug.csv (có thêm ~4000 Human mẫu)
- **R-Drop** cho cả RoBERTa (EN) lẫn PhoBERT (VI)
- Routing logic giữ nguyên: detect ngôn ngữ → hard routing nếu $P \geq 0.8$, soft voting nếu không chắc

```
!pip install transformers torch scikit-learn sentencepiece langdetect underthesea -q
```

```
import pandas as pd
import numpy as np
import torch
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer,
AutoModelForSequenceClassification, get_linear_schedule_with_warmup
from torch.optim import AdamW
from sklearn.metrics import f1_score, classification_report
from sklearn.utils.class_weight import compute_class_weight
from underthesea import word_tokenize
from langdetect import detect_langs, DetectorFactory,
LangDetectException
from pathlib import Path
from tqdm import tqdm
import json, os, subprocess, shutil
```

```
DetectorFactory.seed = 42
torch.manual_seed(42)
np.random.seed(42)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {device}")
```

```
RO_MODEL      = "roberta-base"
PH_MODEL      = "vinai/phobert-base-v2"
RO_OUTPUT     = "models/roberta_rdrop"
PH_OUTPUT     = "models/phobert_rdrop"
TRAIN_CSV     = "/kaggle/input/datasets/cminhnguyndsdsds/faidset-processed/train.csv"
VAL_CSV       = "/kaggle/input/datasets/cminhnguyndsdsds/faidset-
```

```
processed/val.csv"
MAX_LEN      = 128
BATCH_SIZE   = 32
EPOCHS       = 3
LR           = 2e-5
RDROP_ALPHA  = 0.5
LANG_THRESHOLD = 0.8
```

Device: cuda

```
class TextDataset(Dataset):
    def __init__(self, df, tokenizer, max_len, segment=False):
        self.texts = df["text"].tolist()
        self.labels = df["label"].tolist()
        self.tok = tokenizer
        self.max_len = max_len
        self.segment = segment

    def __len__(self): return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]
        if self.segment:
            text = word_tokenize(text, format="text")
            enc = self.tok(text, max_length=self.max_len,
                           padding="max_length", truncation=True,
                           return_tensors="pt")
        return {"input_ids": enc["input_ids"].squeeze(),
                "attention_mask": enc["attention_mask"].squeeze(),
                "label": torch.tensor(self.labels[idx],
                                       dtype=torch.long)}

    def compute_kl_loss(p_logits, q_logits):
        p = F.softmax(p_logits, dim=-1)
        q = F.softmax(q_logits, dim=-1)
        return (F.kl_div(F.log_softmax(p_logits, dim=-1), q,
                        reduction="batchmean") +
                F.kl_div(F.log_softmax(q_logits, dim=-1), p,
                        reduction="batchmean")) / 2

    def train_model_rdrop(model, train_loader, val_loader, output_dir,
                           tokenizer,
                           loss_fn, epochs=EPOCHS, lr=LR,
                           alpha=RDROP_ALPHA):
        optimizer = AdamW(model.parameters(), lr=lr, weight_decay=0.01)
        total_steps = len(train_loader) * epochs
        scheduler = get_linear_schedule_with_warmup(optimizer,
            int(0.1*total_steps), total_steps)
```

```

Path(output_dir).mkdir(parents=True, exist_ok=True)
best_f1 = 0

for epoch in range(epochs):
    model.train()
    total_loss = 0
    for batch in train_loader:
        optimizer.zero_grad()
        ids = batch["input_ids"].to(device)
        mask = batch["attention_mask"].to(device)
        lbls = batch["label"].to(device)

        out1 = model(input_ids=ids, attention_mask=mask)
        out2 = model(input_ids=ids, attention_mask=mask)

        ce_loss = (loss_fn(out1.logits, lbls) +
loss_fn(out2.logits, lbls)) / 2
        kl_loss = compute_kl_loss(out1.logits, out2.logits)
        loss = ce_loss + alpha * kl_loss

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        total_loss += loss.item()

    model.eval()
    preds, trues = [], []
    with torch.no_grad():
        for batch in val_loader:
            out = model(input_ids=batch["input_ids"].to(device),
attention_mask=batch["attention_mask"].to(device))
            preds.extend(out.logits.argmax(-1).cpu().numpy())
            trues.extend(batch["label"].numpy())

    val_f1 = f1_score(trues, preds, average="macro")
    print(f" Epoch {epoch+1}/{epochs} | loss:
{total_loss/len(train_loader):.4f} | val F1: {val_f1:.4f}")
    if val_f1 > best_f1:
        best_f1 = val_f1
        model.save_pretrained(output_dir)
        tokenizer.save_pretrained(output_dir)
        print(f" Saved (F1={best_f1:.4f})")

    return best_f1

print("Utilities ready")

```

Utilities ready

```
train_df = pd.read_csv(TRAIN_CSV, encoding="utf-8-sig")
val_df    = pd.read_csv(VAL_CSV,    encoding="utf-8-sig")
print(f"Train: {len(train_df):,} | Val: {len(val_df):,}")
```

Train: 48,257 | Val: 4,920

Phần 1 — Train RoBERTa (EN) + R-Drop

```
train_en = train_df[train_df["language"]=="en"].reset_index(drop=True)
val_en    = val_df[val_df["language"]=="en"].reset_index(drop=True)
print(f"[EN] Train: {len(train_en):,} | Val: {len(val_en):,}")
```

```
tok_ro    = AutoTokenizer.from_pretrained(RO_MODEL)
model_ro  =
AutoModelForSequenceClassification.from_pretrained(RO_MODEL,
num_labels=2).to(device)
```

```
train_loader_en = DataLoader(TextDataset(train_en, tok_ro, MAX_LEN),
batch_size=BATCH_SIZE, shuffle=True)
val_loader_en    = DataLoader(TextDataset(val_en,    tok_ro, MAX_LEN),
batch_size=BATCH_SIZE)
```

```
cw_en = compute_class_weight("balanced", classes=np.array([0,1]),
y=train_en["label"].values)
loss_fn_ro = torch.nn.CrossEntropyLoss(weight=torch.tensor(cw_en,
dtype=torch.float).to(device))
```

```
print("\n Training RoBERTa (EN) + R-Drop ")
best_ro = train_model_rdrop(model_ro, train_loader_en, val_loader_en,
RO_OUTPUT, tok_ro, loss_fn_ro)
print(f"\n RoBERTa done. Best val F1: {best_ro:.4f}")
del model_ro; torch.cuda.empty_cache()
```

[EN] Train: 29,216 | Val: 3,025

Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.

Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.

```
{"model_id": "848dac87ba6146f3a59ab8860dff1129", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "016026c4ec174d71b59458d65ac27106", "version_major": 2, "version_minor": 0}
```

```

{"model_id": "830a6c2a55154a3baeca0fb0033a87f4", "version_major": 2, "version_minor": 0}

{"model_id": "6753bc95a1ab4443a4bcd094ee405cb", "version_major": 2, "version_minor": 0}

{"model_id": "d4f3af3e355d41c29eac4228b2ca4f0d", "version_major": 2, "version_minor": 0}

{"model_id": "be13b7bc8bec4763b52c17e8b1ae5975", "version_major": 2, "version_minor": 0}

{"model_id": "a9d016699a964a958af940c547e52377", "version_major": 2, "version_minor": 0}

```

RobertaForSequenceClassification LOAD REPORT from: roberta-base

Key	Status
lm_head.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
roberta.embeddings.position_ids	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
classifier.dense.weight	MISSING
classifier.dense.bias	MISSING
classifier.out_proj.weight	MISSING
classifier.out_proj.bias	MISSING

Notes:

- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING : those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Training RoBERTa (EN) + R-Drop

Epoch 1/3 | loss: 0.3450 | val F1: 0.8429

```

{"model_id": "5c771ff7e975429887cceb6d3ed98feb", "version_major": 2, "version_minor": 0}

```

Saved (F1=0.8429)

Epoch 2/3 | loss: 0.1441 | val F1: 0.8834

```

{"model_id": "5d6b933efaf74fe9a2bac1836e6218b0", "version_major": 2, "version_minor": 0}

```

Saved (F1=0.8834)

Epoch 3/3 | loss: 0.0732 | val F1: 0.8834

```
{"model_id": "d9587a75891a4587ab40376120925873", "version_major": 2, "version_minor": 0}
```

Saved (F1=0.8834)

RoBERTa done. Best val F1: 0.8834

Phần 2 — Train PhoBERT (VI) + R-Drop

```
train_vi = train_df[train_df["language"]=="vi"].reset_index(drop=True)
val_vi = val_df[val_df["language"]=="vi"].reset_index(drop=True)
print(f"[VI] Train: {len(train_vi):,} | Val: {len(val_vi):,}")
print("DataLoader segment on-the-fly với underthesea")
```

```
tok_ph = AutoTokenizer.from_pretrained(PH_MODEL)
model_ph =
AutoModelForSequenceClassification.from_pretrained(PH_MODEL,
num_labels=2).to(device)
```

```
train_loader_vi = DataLoader(TextDataset(train_vi, tok_ph, MAX_LEN,
segment=True), batch_size=BATCH_SIZE, shuffle=True)
val_loader_vi = DataLoader(TextDataset(val_vi, tok_ph, MAX_LEN,
segment=True), batch_size=BATCH_SIZE)
```

```
# VI đã F1≈0.99 baseline → không dùng class weight
loss_fn_ph = torch.nn.CrossEntropyLoss()
```

```
print("\n Training PhoBERT (VI) + R-Drop ")
best_ph = train_model_rdrop(model_ph, train_loader_vi, val_loader_vi,
PH_OUTPUT, tok_ph, loss_fn_ph)
print(f"\n PhoBERT done. Best val F1: {best_ph:.4f}")
del model_ph; torch.cuda.empty_cache()
```

```
[VI] Train: 19,041 | Val: 1,895
DataLoader segment on-the-fly với underthesea
```

```
{"model_id": "c6dd21a44ebd475f9f6c5876fc570c1d", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "cb775c6b1de84bc3bd6fa8f57feee1a2", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "7030d0e18b7c49f2b8f3be8311e756fb", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "5b6eae62794442838aaa4d621d0d6451", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "24dbdc0cedb84b87831a0042a60480cf", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d59bde2f6dae49dea372c0b3e2495ed6", "version_major": 2, "version_minor": 0}
```

RobertaForSequenceClassification LOAD REPORT from: vinai/phobert-base-v2

Key	Status
lm_head.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
roberta.embeddings.position_ids	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
classifier.dense.weight	MISSING
classifier.dense.bias	MISSING
classifier.out_proj.weight	MISSING
classifier.out_proj.bias	MISSING

Notes:

- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING : those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Training PhoBERT (VI) + R-Drop

```
{"model_id": "41459d41857c42648e0525b886ce6aa5", "version_major": 2, "version_minor": 0}
```

Epoch 1/3 | loss: 0.1056 | val F1: 0.9698

```
{"model_id": "8b5a00a481be401a80b8e68e3fce3b38", "version_major": 2, "version_minor": 0}
```

Saved (F1=0.9698)

Epoch 2/3 | loss: 0.0141 | val F1: 0.9778

```
{"model_id": "43bb1d2a359846009a184325a0585558", "version_major": 2, "version_minor": 0}
```

Saved (F1=0.9778)

Epoch 3/3 | loss: 0.0054 | val F1: 0.9892

```
{"model_id": "4ef2523039354a0c8531746bca3b620b", "version_major": 2, "version_minor": 0}
```

Saved (F1=0.9892)

PhoBERT done. Best val F1: 0.9892

Phần 3 — Đánh giá Hybrid inference trên val set

```
model_ro_best =
AutoModelForSequenceClassification.from_pretrained(RO_OUTPUT).to(device).eval()
tok_ro_best = AutoTokenizer.from_pretrained(RO_OUTPUT)
model_ph_best =
AutoModelForSequenceClassification.from_pretrained(PH_OUTPUT).to(device).eval()
tok_ph_best = AutoTokenizer.from_pretrained(PH_OUTPUT)

def detect_language(text):
    try:
        res = detect_langs(text[:500])
        top = res[0]
        return (top.lang if top.lang in ["en", "vi"] else "other"),
        top.prob
    except LangDetectException:
        return "other", 0.0

def predict_proba(text, model, tokenizer, segment=False):
    if segment: text = word_tokenize(text, format="text")
    enc = tokenizer(text, max_length=MAX_LEN, padding="max_length",
                    truncation=True, return_tensors="pt")
    with torch.no_grad():
        logits = model(input_ids=enc["input_ids"].to(device),
                        attention_mask=enc["attention_mask"].to(device)).logits
        return F.softmax(logits, dim=-1)[0][1].item()

def hybrid_predict(text):
    lang, prob = detect_language(text)
    if prob >= LANG_THRESHOLD:
        if lang == "en": p_ai = predict_proba(text, model_ro_best,
        tok_ro_best)
        elif lang == "vi": p_ai = predict_proba(text, model_ph_best,
        tok_ph_best, segment=True)
        else: p_ai = (predict_proba(text, model_ro_best,
        tok_ro_best) +
                    predict_proba(text, model_ph_best,
        tok_ph_best, segment=True)) / 2
    else:
        p_ai = (predict_proba(text, model_ro_best, tok_ro_best) +
                predict_proba(text, model_ph_best, tok_ph_best,
        segment=True)) / 2
    return 1 if p_ai >= 0.5 else 0

print("Evaluating Hybrid on val set...")
val_preds = [hybrid_predict(t) for t in tqdm(val_df["text"].tolist())]
val_f1_hybrid = f1_score(val_df["label"].tolist(), val_preds,
```



```

average="macro")
print(f"\nHybrid val F1: {val_f1_hybrid:.4f}")
print(classification_report(val_df["label"].tolist(), val_preds,
target_names=["Human", "AI"]))

{"model_id": "f1d0fd7279de46ab95d0a0c72756b801", "version_major": 2, "version_minor": 0}

{"model_id": "61dc82711882482b89e2128ba0dc4dc7", "version_major": 2, "version_minor": 0}

```

Evaluating Hybrid on val set...

100%|██████████| 4920/4920 [01:09<00:00, 71.14it/s]

Hybrid val F1: 0.9266

	precision	recall	f1-score	support
Human	0.98	0.87	0.92	2470
AI	0.88	0.98	0.93	2450
accuracy			0.93	4920
macro avg	0.93	0.93	0.93	4920
weighted avg	0.93	0.93	0.93	4920

```

# Upload ca' 2 model lên 1 dataset
dataset_name = "hybrid-rdrop"
upload_dir = "hybrid_upload"
os.makedirs(f"{upload_dir}/roberta", exist_ok=True)
os.makedirs(f"{upload_dir}/phobert", exist_ok=True)
for fn in os.listdir(R0_OUTPUT):
    shutil.copy2(f"{R0_OUTPUT}/{fn}", f"{upload_dir}/roberta/{fn}")
for fn in os.listdir(PH_OUTPUT):
    shutil.copy2(f"{PH_OUTPUT}/{fn}", f"{upload_dir}/phobert/{fn}")

kaggle_user = [l.split(":")[1].strip() for l in
                subprocess.run("kaggle config view", shell=True,
capture_output=True, text=True)
                .stdout.split("\n") if "username" in l][0]

with open(f"{upload_dir}/dataset-metadata.json", "w") as f:
    json.dump({"title": dataset_name, "id":
f"{kaggle_user}/{dataset_name}",
               "licenses": [{"name": "CC0-1.0"}]}, f)

check = subprocess.run(f"kaggle datasets list --user {kaggle_user} --
search {dataset_name}",
                        shell=True, capture_output=True, text=True)

```

```
cmd = (f'kaggle datasets version -p {upload_dir} -m "update" --dir-  
mode zip'  
      if dataset_name in check.stdout  
      else f"kaggle datasets create -p {upload_dir} --dir-mode zip")  
print(subprocess.run(cmd, shell=True, capture_output=True,  
text=True).stdout)  
print(f"Done! {kaggle_user}/{dataset_name}")
```

Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.1), please consider upgrading to the latest version
(2.0.2)

Starting upload for file phobert.zip

Upload successful: phobert.zip (479MB)

Starting upload for file roberta.zip

Upload successful: roberta.zip (436MB)

Your private Dataset is being created. Please check progress at
<https://www.kaggle.com/datasets/minhbodoi/hybrid-rdrop>

Done! minhbodoi/hybrid-rdrop